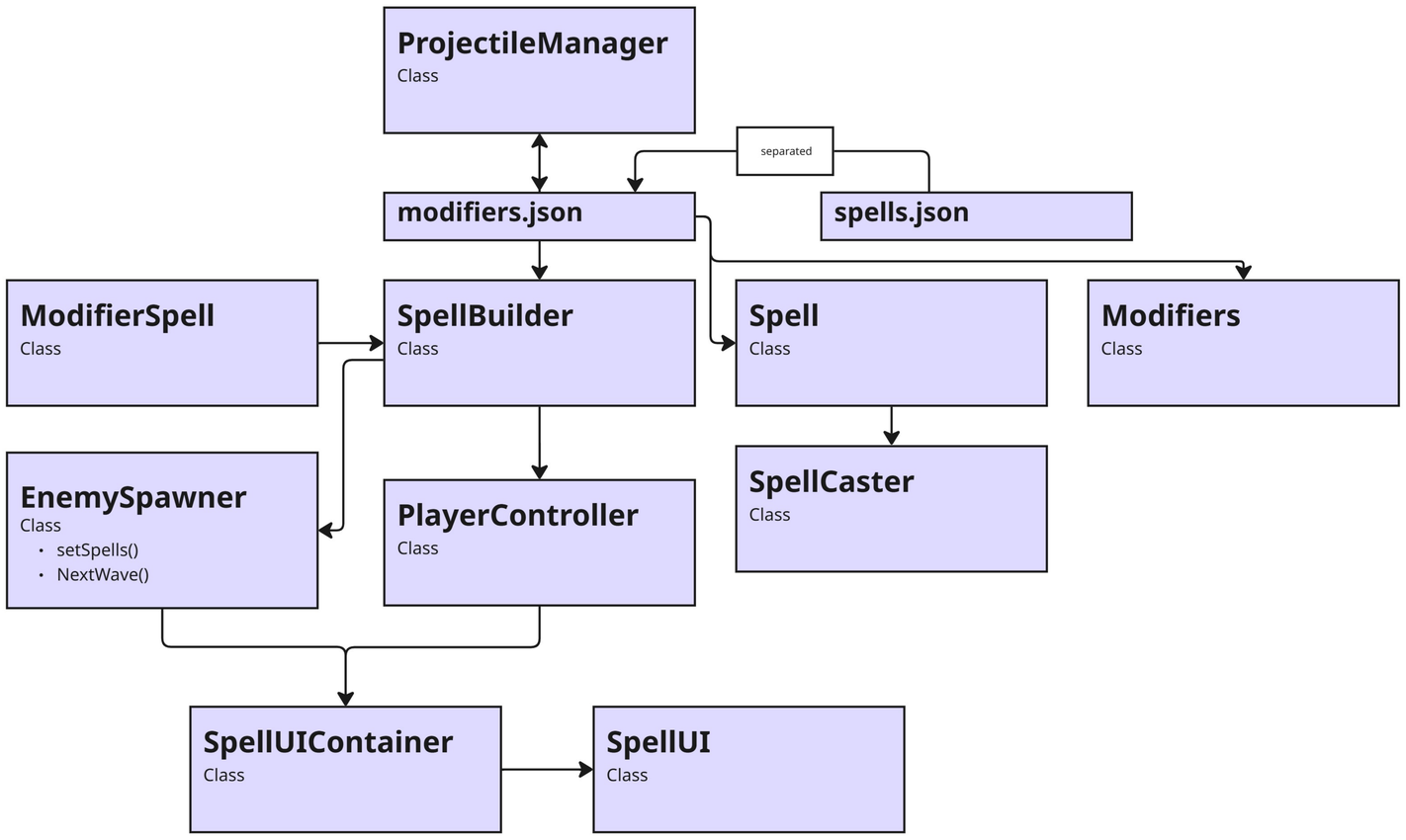

Architecture

We separated modifiers from spells and read them separately. Spell.cs contains a namespace with default values for all the modifiers, which the different if statements within ModifierSpell.cs overwrite if they're present in the modifier list randomly generated by SpellBuilder.cs every time a new spell gets created by setSpells() and called by NextWave() in EnemySpawner.cs. For our modifiers "octupled" and "whirlpool," we added a new parameter in modifiers.json called 'count.' A for loop in Spell.cs's Cast method creates a new projectile based on this count.

Sai's Paragraph

I worked on implementing the modifiers & working on the SpellBuilder to generate new spells with 2-4 modifiers each in each wave. The modifier spells we added were 1) firerate-amplified, which decreased the damage but also decreased the cooldown of the projectiles, 2) octupled, which cast the projectile in 8 directions at an increased mana cost, and 3) whirlpool, in which 4 projectiles were sent spiraling at the cost of an increased cooldown. It was kind of annoying to figure out how to implement multiple projectiles on input, but once I understood the interaction between projectile spawning and coroutines I was able to use a for loop in Spell.cs to make it happen. I learned a lot about how different parts of a game codebase reference each other; I was hopping around a lot of different scripts in order to get the modifiers to function properly and this was just very obviously the most complicated functionality I've felt responsible for before (I've made a lot of games but I'm generally working in art/design/production; most of my in engine work is tuning or asset implementation). I'm sorry for how long it took to get this going; the last few weeks were absolutely hellish but we finally got it done.

Johnny's Paragraph

I did these three parts for this assignment
- Add functionality to show the player the potential spell reward they may get to choose after each wave, and allow them to take it (and drop existing spells if necessary)
- Define **at least** three additional modifier spells, where **at least** one adds behavior beyond mere stats changes
- Scale player stats with the wave number

Adding in the UI was the hardest part. Because of the schedule difficulties, I ended up doing the first half 2 weeks before I did the second half. To get player progression I changed around the StartLevel function in the player controller, which then made it hard to figure out how to switch between the spells. Once I understood exactly how Sai's Spellbuilder and modifer classes worked, as well as the spellcaster class, it was a lot easier to figure out how to store spells and swap between them when necessary. I was very confused on how to change the spell icon as well before I realized that Markus had already implemented that in SpellUI (Thank god!). Sorry this took so long to get out but im happy with the end result! I'm finally starting to get the hang of Unity.